

THE DOTS

macOS Product + Technical Architecture

MVP implementation brief for Codex and Grok · revised after the first shell prototype



Purpose. This document is the source of truth for The Dots. Coding agents should implement the smallest native macOS system that satisfies the behavior below. Do not expand the product into a general launcher, note app, calendar, full clipboard manager, or automation platform unless explicitly requested.

Product principle	Meaning
One click away	A Dot is always accessible at the top edge of the screen.
Do the thing	Each feature performs one small job with minimal UI.
Disappear	Panels and overlays dismiss immediately when the job is done.
User chooses the set	The app may offer many features, but only 1–5 active Dots are visible.
Native first	Prefer Apple frameworks, local processing, and no backend for MVP.
Orbs never grow	The 16 pt Dots stay 16 pt. Feature UI hangs below or detaches; it does not morph the orb.

Prototype status (September 2026). The tree at yaosamo/dots is a working top-edge shell: registry-backed Mirror + text Tasks, stable-anchor nearest-Dot magnetism, nonactivating panel, click-through around orbs, and a draggable 120 pt Mirror preview. It is **not** the full MVP. Escape/presenter completion, multi-display behavior, Pomodoro, Red Pen, Screen → Text, Clipboard, voice tasks, and onboarding are still required. Unimplemented catalog Dots are not shown as decoration.

Target platform: macOS. Minimum: macOS 14. Enhanced local AI (Foundation Models, modern Speech APIs) is availability-gated on newer systems.

1. Product model

The Dots is a compact top-edge utility layer. The visible objects are black circular endpoints attached to short sprouts hanging from the top edge of the current display. The user configures which utilities are present. The shell is persistent; each utility is deliberately tiny.

1.1 Launcher behavior

- Show 1–5 active Dots. Five is the MVP maximum; keep the limit a named constant in code.
- Default visual orb diameter: 16 × 16 pt. The interactive hit area must be larger than the visible orb (stem hit slop plus a padded circle), not equal to the 16 pt fill.
- On pointer movement near the launcher, calculate distance from the pointer to each **stable logical orb center**. Only the nearest eligible orb reacts.
- The nearest sprout/orb stretches toward the pointer with a short ease-out (start ~60–100 ms). Selection uses the unstretched anchor so the animation cannot jitter hit-testing.

- Clicking an orb opens that feature. Clicking it again or pressing Escape dismisses it and returns control to the prior app.
- Feature UI appears adjacent to its Dot. It is not a traditional app window. Become key only when keyboard input is required (Tasks composer).
- One transient feature at a time. Opening Tasks closes Mirror, and the other way around.
- Anchor to the active display under the pointer. Place the cluster flush with `visibleFrame.maxY` (just below the menu bar / notch) and horizontally centered. Keep per-display geometry isolated; do not assume one global coordinate space.
- The cluster itself is not user-draggable in MVP. The Mirror **preview** is independently draggable (see 5.1).

1.2 Onboarding

The first launch should double as configuration. Present the Dots themselves as an interactive selector: hovering or focusing a Dot reveals a compact preview/name, and selecting it adds the utility to the user's top-edge set. Avoid a long settings wizard.

Recommended first-run defaults: Mirror, Tasks, Pomodoro.

After onboarding, configuration remains available through a small settings view: add/remove/reorder Dots, timer defaults, clipboard history length, and permissions status.

The current prototype launches with the implemented Mirror and Tasks descriptors. Add catalog Dots to the visible set only when their behavior is implemented; do not ship unused decorative Dots.

1.3 UX invariants

Invariant	Acceptance rule
No app chrome	No title bar, sidebar, document navigation, or persistent large window in normal use. No Dock icon (LSUIElement).
Fast open	A click should produce visible response immediately; expensive work starts after the panel/overlay is visible.
Keyboard-safe	Non-text features must not steal focus from the currently active app. Use a borderless nonactivating <code>NSPanel</code> . <code>canBecomeMain</code> is false. Become key only for the Tasks composer.
Escape means exit	Escape closes any Dot panel or transient overlay and returns control to the prior app.
Orbs stay 16 pt	Camera, lists, and other widgets hang below the row or float free. They never enlarge the orb.
State survives appropriately	Tasks and configuration persist. Timer survives app UI dismissal. Mirror frames, Red Pen drawings, OCR selections, and clipboard history do not require durable storage in MVP.

2. MVP feature set

Dot	Job	MVP behavior	Now
Mirror	Quickly check camera view	Open a mirrored live camera preview in a 120 pt circle hanging from the orb. Drag it anywhere on the display. When it leaves the docked hang point, that orb's sprout disappears. Close re-docks. No recording or photo capture.	Built
Tasks	Capture things to do	Small checklist. Add text with one field + Return or +. Persist locally. Voice: dictate one utterance that can become multiple tasks (not in the tree yet).	Text only
Pomodoro	Start a focus timer	Quick presets (5 / 15 / 25) plus start/pause/cancel. Persist deadline rather than ticking state.	Not built

Dot	Job	MVP behavior	Now
Red Pen	Point/draw over the screen	Temporary transparent overlay for pen / highlighter / arrow. Escape clears and exits.	Not built
Screen → Text	Copy visible text from anywhere	Select a rectangle, OCR the captured pixels, copy recognized text, show a brief confirmation.	Not built
Clipboard	Recover recent copied text	Keep the most recent 5–10 text/URL clipboard entries. Click an item to copy it again.	Not built

Not in MVP. Calendar integration, recent-files browser, password vault, full note system, cloud sync, accounts, team features, plugin marketplace, cross-device sync, clipboard search, annotation saving, screen recording, or analytics-heavy dashboards. No feature should create its own Dock-level application experience. Everything is subordinate to the top-edge shell.

3. System architecture

Use a SwiftUI + AppKit hybrid. SwiftUI owns feature views and lightweight state. AppKit owns window/panel behavior, top-edge positioning, nonactivating interaction, transparent overlays, and screen geometry. Keep feature implementations isolated behind a common registry so additional Dots can be added without modifying the launcher core.

The current tree is a flat prototype (AppDelegate, DotsView, CameraSession, TaskStore). That is acceptable through checkpoint 3. Starting at checkpoint 2 (launcher system), introduce the registry and presenter so Pomodoro and later Dots do not keep growing DotsView.

```
TheDotsApp
AppCoordinator
|-- DotRegistry ----- metadata + factories
|-- LauncherController ----- top-edge NSPanel / geometry / hover
|-- FeaturePresenter ----- opens transient feature panels
|-- PermissionCenter ----- camera / mic / speech / screen capture
|
+-- Features
    |-- Mirror
    |-- Tasks
    |-- Pomodoro
    |-- RedPen
    |-- ScreenToText
    +-- Clipboard

Shared services
CameraService | SpeechService | TaskExtractionService
ScreenCaptureService | OCRService | ClipboardService
TimerService | SettingsStore | TaskStore
```

3.1 Core boundaries

Component	Responsibility
AppCoordinator	Bootstraps services, restores configuration, owns lifecycle, routes Dot actions.
DotRegistry	Static catalog of available Dots: id, display name, permissions, preferred presentation mode, factory/action.
LauncherController	Owns the always-on-top top-edge NSPanel, pointer-distance logic, animations, ordering, display changes.
FeaturePresenter	Creates/dismisses transient NSPanel windows and full-screen transparent overlays. Enforces one active transient feature at a time.
PermissionCenter	Single place to query/request permissions and expose permission state to onboarding/settings.

Component	Responsibility
Feature modules	Contain feature-specific view/state and use shared services through protocols.

3.2 Suggested feature abstraction

```
enum DotPresentation { case popover, overlay, directAction }

struct DotDescriptor: Identifiable {
    let id: DotID
    let title: String
    let presentation: DotPresentation
    let requiredPermissions: Set<AppPermission>
}

protocol DotFeature {
    var descriptor: DotDescriptor { get }
    @MainActor func activate(context: DotContext) async
    @MainActor func deactivate()
}

struct DotContext {
    let anchorRect: CGRect
    let screen: NSScreen
    let services: AppServices
}
```

Do not force every feature into the same UI container. Mirror, Tasks, Pomodoro, and Clipboard are panel-like. Red Pen is an overlay. Screen → Text is a direct action that temporarily creates a selection overlay and then exits.

4. Windowing and interaction architecture

The launcher is an AppKit NSPanel at the top of the active screen. Style mask: `borderless + nonactivatingPanel`. Hovering and clicking Dots must not pull the user out of the current app. A feature panel may become key only when it actually needs keyboard entry.

4.1 Launcher panel

- Clear background, no shadow unless the final design needs one.
- Window level high enough to remain visible over normal windows; test around full-screen apps, Mission Control, menu bar, and system alerts. Current prototype uses `.statusBar` with `.canJoinAllSpaces`, `.fullScreenAuxiliary`, `.stationary`, `.ignoresCycle`.
- `becomesKeyOnlyIfNeeded = true`. `canBecomeKey = true`. `canBecomeMain = false`.
- Hit-test must return nil for transparent padding so clicks pass through to apps underneath. Resize the panel to the union of the orb row and any hanging/detached widget; do not cover the whole screen.
- SwiftUI launcher content is embedded with `NSHostingView`.
- Right-click a Dot for Quit Dots. `Cmd+Q` works after the panel is key.

4.2 Magnetic sprout interaction

Required. The prototype only scales the hovered orb by 4%; that is not magnetism.

```
onPointerMove(point):
    candidates = visibleDots.map { (dot, distance(point, dot.orbCenter)) }
    nearest = candidates.min(by: distance)

    for dot in visibleDots:
        if dot == nearest && distance < activationRadius:
            dot.targetOffset = clampedVectorTowardPointer(point)
        else:
            dot.targetOffset = .zero

    animate(targetOffset, duration: 0.06...0.10, easing: easeOut)
```

The orb may visually stretch toward the pointer, but selection remains based on stable logical anchor centers.

4.3 Presentation rules

Feature type	Window behavior
Panel	Anchor below the selected Dot. Dismiss on outside click where practical, Escape, or second click. Mirror's preview may then be dragged off the hang point.
Overlay	Transparent borderless window spanning the relevant display(s). Escape always exits. Keep an explicit event mode so the overlay only intercepts input while active.
Direct action	Show selection/capture UI only for the duration of the action, then a short toast and disappear.

5. Feature technical specifications

5.1 Mirror

Goal: one-click live mirror, not a camera app.

- AVFoundation capture session with a preferred built-in front camera, then Continuity Camera, then external. Mirrored preview via AVCaptureVideoPreviewLayer.
- Render in a 120 pt circle that hangs below the Mirror orb with an 8 pt gap. Orbs stay 16 pt.
- The preview is draggable. The user may place it anywhere on the current display. Store the offset from the docked hang point; the launcher panel grows to the union of the row and the preview and stays click-through around both.
- When the preview moves farther than 8 pt from the docked hang point, the Mirror orb's sprout disappears. The 16 pt orb remains so the user can close Mirror. Closing re-docks the preview (offset resets to zero) and restores the sprout.
- Start the session after the panel opens; stop immediately on dismiss. Serialize start/stop with a generation token so a queued idle cannot overwrite a newer open. Recheck the token after startRunning().
- Prefer a 640×480 session preset (then medium / low). Do not use .high for a 120 pt circle.
- On media-services reset while the preview is open, retry once. Other runtime errors fail until the user toggles the orb. Connecting a camera later is click-to-retry, not hot-plug auto-start.
- No photo/video saving. No microphone. Request camera permission only when the user first activates Mirror or enables it in onboarding.

5.2 Tasks

Goal: a tiny checklist with fast text and voice capture. A task is an action item, not a general note.

- MVP persistence: local UserDefaults JSON is acceptable. Model: id, title, isDone. Add createdAt / completedAt / sortOrder when the UI needs them. SwiftData is optional later if the model grows.

- No due dates, projects, tags, reminders, collaboration, or recurring tasks.
- Text entry: one AppKit text field (the panel is a nonactivating overlay; SwiftUI `TextField` does not reliably take keys) + Return or + to create. Opening Tasks makes the panel key and focuses the composer. After insert, clear the field even if it is still editing.
- Completed items can collapse into a small completed section (not in the prototype yet).
- Voice entry (required for MVP, not built): press/hold or tap a mic affordance, dictate naturally, transcribe, convert the transcript into one or more tasks.

```
struct DotTask: Identifiable, Codable, Equatable {
    var id: UUID
    var title: String
    var isDone: Bool
}

struct TaskDraft: Codable, Sendable {
    let text: String
}
```

Voice pipeline (still to build)

- Capture microphone audio.
- Transcribe with Apple Speech APIs. On newer systems prefer `SpeechAnalyzer` / `SpeechTranscriber`; keep a compatibility adapter for older supported macOS if needed.
- Send the transcript to `TaskExtractionService`.
- If Apple Foundation Models is available, use structured/guided generation to return `[TaskDraft]`.
- If the local model is unavailable or unsupported for the current language, use a deterministic fallback splitter. A cloud model adapter can be added later, but must not be required for MVP.
- Show the parsed tasks for immediate confirmation/editing, then persist.

Task extraction contract

```
Input: "Finish the website, send the mockups to Alex, and buy cat food."
Output: [
    { "text": "Finish the website" },
    { "text": "Send the mockups to Alex" },
    { "text": "Buy cat food" }
]

Rules:
- extract actions only; do not invent tasks
- preserve names and concrete details
- split clearly separate actions
- do not add dates or priorities unless explicitly spoken
```

5.3 Pomodoro / Timer

- Preset buttons: 5, 15, and 25 minutes; make presets user-configurable later.
- Store `startedAt` and `targetEndDate`. UI derives remaining time from wall clock instead of decrementing a counter every second.
- Closing the panel must not stop the timer.
- When `targetEndDate` is reached, send a local notification if permission is granted and show the Dot in a completed/attention state.

- MVP does not need productivity statistics, sessions history, projects, or cloud sync.

5.4 Red Pen

Goal: temporary visual annotation over whatever is already on screen.

- Activate a transparent overlay window over the current display; optionally support all displays after single-display behavior is solid.
- MVP tools: red pen, translucent highlighter, arrow. Keep the tool strip minimal or reveal it near the active Dot.
- Pointer events are intercepted only while Red Pen is active. Underlying apps must receive input again immediately after exit.
- Escape clears all strokes and closes the overlay. Cmd+Z may undo the last stroke.
- Do not save annotations. Do not require Screen Recording permission merely to draw an overlay.
- Represent strokes as lightweight vector paths in memory, not screenshots.

5.5 Screen → Text

Goal: two interactions — click the Dot, drag a rectangle, then the recognized text is already in the clipboard.

- Activate a selection overlay and let the user drag a rectangle.
- Translate selection coordinates into the correct display coordinate space.
- Capture only the selected region with `ScreenCaptureKit` (`sourceRect` where appropriate).
- Pass the resulting image to `VisionRecognizeTextRequest`. Prefer accurate recognition; allow language hints from system locale.
- Join recognized lines in visual order, trim empty lines, write the final string to `NSPasteboard.general`.
- Show a small “Copied” confirmation for about 1 second, then exit.
- Screen capture requires Screen Recording permission. Denial should show a compact explanation with a button to open System Settings, not a broken selection flow.

5.6 Clipboard

Goal: recover the last few copied text snippets without becoming a full clipboard product.

- Poll `NSPasteboard.general.changeCount` on a modest interval while the app is running. When it changes, read supported content types.
- MVP stores only plain text and URLs. Default history size: 10; make 5–10 configurable.
- Deduplicate consecutive identical values. Clicking a history item writes it back to the pasteboard and closes the panel.
- Keep clipboard history in memory only. Do not persist it across launches.
- Avoid capturing data marked transient/concealed by source apps when detectable. Add an explicit “Pause clipboard history” switch.
- Never market this as password storage. A future secure-snippets feature would be a separate design using Keychain.

6. State, persistence, and services

Data	Storage	Notes
Active Dot configuration	UserDefaults / @AppStorage	Ordered list of DotIDs, max count, launcher preferences.

Data	Storage	Notes
Tasks	UserDefaults JSON for MVP	Durable user content. SwiftData later only if the model grows.
Timer	UserDefaults	Persist targetEndDate/status so UI can reconstruct after dismissal/relaunch.
Clipboard history	Memory only	Privacy-first MVP; cleared on quit.
Red Pen strokes	Memory only	Destroyed on exit.
OCR image	Memory only	Release immediately after recognition.
Mirror frames	Not stored	Live preview only. Preview offset is session-only; close re-docks.

6.1 Service protocols

```
protocol SpeechTranscribing {
    func transcribeLive() -> AsyncThrowingStream<String, Error>
    func stop() async
}

protocol TaskExtracting {
    func extractTasks(from transcript: String) async throws -> [TaskDraft]
}

protocol OCRRecognizing {
    func recognizeText(in image: CGImage) async throws -> String
}

protocol ScreenRegionCapturing {
    func capture(rect: CGRect, on screen: NSScreen) async throws -> CGImage
}

protocol ClipboardMonitoring {
    var items: AsyncStream<[ClipboardItem]> { get }
    func copy(_ item: ClipboardItem)
}
```

Keep Apple-framework types inside service implementations where practical. Feature views should depend on app protocols.

7. Permissions, sandboxing, and privacy

Capability	Used by	Behavior
Camera	Mirror	Request on first use; camera usage description + sandbox camera entitlement. Present in the prototype.
Microphone	Voice tasks	Request on first use; microphone usage description + Audio Input entitlement.
Speech recognition	Voice tasks	Provide required usage description/authorization path for the Speech framework APIs used.
Screen Recording	Screen → Text	ScreenCaptureKit requires user authorization. Detect denial and provide recovery UI.
Notifications	Pomodoro	Optional. Timer still functions if notification permission is denied.
Accessibility / Input Monitoring	None in base MVP	Do not add unless later functionality truly requires synthetic input/global event taps.

Privacy defaults. No backend for the base MVP. Prefer on-device OCR, speech, and task extraction when available. Do not retain camera frames, microphone recordings, OCR screenshots, or Red Pen pixels. If a future cloud LLM fallback is added, it must be an explicit provider layer with disclosure and opt-in. Clipboard history is in-memory and easy to pause/clear.

8. Suggested Xcode project structure

Target layout as the registry lands. The prototype may stay flatter until checkpoint 2.

```
TheDots/
App/      TheDotsApp, AppCoordinator, AppServices
Core/     DotID, DotDescriptor, DotRegistry, DotContext,
          PermissionCenter, SettingsStore
Launcher/ LauncherController, LauncherPanel, LauncherView,
          DotOrbView, DotMagnetism, ScreenGeometry
Presentation/ FeaturePresenter, TransientPanel, OverlayWindow,
              ToastPresenter
Features/    Mirror, Tasks, Pomodoro, RedPen, ScreenToText, Clipboard
Services/    Camera, Speech, TaskExtraction, FoundationModel extractor,
              Fallback extractor, ScreenCapture, OCR, Clipboard, Timer
Data/       TaskItem, TimerState
Settings/   Onboarding/ Resources/

TheDotsTests/
DotMagnetismTests, TaskExtractionTests, TimerStateTests,
ClipboardDedupTests, OCRTextOrderingTests,
DotsLayoutTests (orb geometry, hanging widgets, Mirror detach)
```

9. Implementation sequence

Build vertically in small checkpoints. Do not implement all services before the shell is usable. Keep the project buildable at every checkpoint. No third-party dependencies unless a native implementation is materially worse and the dependency is explicitly approved.

#	Checkpoint	Status
1	Shell prototype: top-edge nonactivating NSPanel, orbs, correct positioning on one display, hover/click, click-through.	Done
2	Launcher system: 1–5 Dots, ordering, nearest-dot magnetism, feature presenter, Escape dismissal, multi-display positioning.	In progress (registry + magnetism)
3	Mirror: permission + mirrored preview + stop on close. Draggable 120 pt circle; sprout hides when detached; generation-token lifecycle; 640×480 preset.	Done (drag included)
4	Pomodoro: timer state outside the panel, notification optional. Validates persistent background-ish state.	Not started
5	Tasks text-only (local persist, AppKit composer). Then voice transcription and task extraction as separate adapters.	Text done; voice not started
6	Screen → Text: selection overlay → region capture → Vision OCR → clipboard → toast.	Not started
7	Clipboard: changeCount monitor, memory history, dedupe, re-copy.	Not started
8	Red Pen: transparent overlay, vector strokes, Escape/undo, pointer-mode restoration.	Not started
9	Onboarding/settings: choose/reorder Dots, permissions status, defaults.	Not started

#	Checkpoint	Status
10	Polish: magnetism animation, reduced-motion, VoiceOver, launch-at-login if desired, App Store sandbox testing.	Partial (reduce-motion on open, VO labels on existing orbs)

10. Acceptance checklist

Area	Pass condition	Now
Launcher	1–5 Dots at top edge, nearest Dot reacts magnetically, no visual jitter, clicks target reliably despite 16 pt orb.	Partial — registry, stable magnetism, padded hits; setting s/order pending
Focus	Hovering/clicking non-text Dots does not activate The Dots or steal keyboard focus.	Pass for non-text (prototype)
Escape	Escape closes the open feature and returns to the prior app.	Fail
Mirror	Preview appears after permission, mirrored, camera stops when closed.	Pass
Mirror drag	Preview can be placed anywhere on the display; Mirror sprout disappears once detached; close re-docks.	Pass
Tasks text	Text task survives relaunch. Composer accepts typing in the overlay and clears after Return/+.	Pass
Tasks voice	One dictated sentence with three clear actions can become three editable tasks.	Fail
Pomodoro	Timer continues while panel is closed and recovers remaining time after UI changes/relaunch.	Fail
Red Pen	Can draw/highlight/arrow over another app; Escape removes everything and returns normal clicking immediately.	Fail
Screen → Text	Drag area → OCR → clipboard with no save dialog or intermediate window.	Fail
Clipboard	Recent 5–10 text items appear, duplicates do not pile up, selecting one copies it again.	Fail
Permissions	Every denied permission fails gracefully and explains the missing capability.	Mirror only
Privacy	No images/audio/clipboard history accidentally written to durable storage.	Pass so far

Performance targets. Launcher idle CPU should be effectively negligible. Pointer tracking should not trigger expensive SwiftUI rebuilds. Feature panel animation should begin immediately after click; do not block UI on permission or model initialization. Prewarm local task extraction only when useful. Clipboard polling around 0.5–1.0 seconds, then measure.

11. Explicit decisions and open questions

Decision	Current choice
Maximum visible Dots	5 for MVP; configurable constant.
Visible orb size	16 × 16 pt; hit area larger than the fill.
Orbs vs widgets	Orbs never grow. Widgets hang below or detach.
Minimum macOS	14.0. Foundation Models and modern Speech APIs are availability-gated.
Backend	None required for MVP.
Task persistence	UserDefaults JSON for MVP. SwiftData later only if needed.
Task AI	Apple Foundation Models when available; deterministic fallback; cloud later if needed.
Clipboard persistence	Memory only.
Red Pen persistence	None.
Mirror persistence	None. Drag offset is session-only; close re-docks.
Cluster drag	No. Only the Mirror preview is draggable.
Launcher placement	Flush with visibleFrame.maxY, horizontally centered, display under the pointer.
Notes feature	Not included.

Still open after the prototype

- Exact magnetic activation radius and easing curve.
- Whether the launcher relocates live as the pointer moves to another display, or only at launch / screen-parameter changes.
- Whether voice task capture is tap-to-toggle or press-and-hold.
- Whether Screen → Text supports multi-line formatting or always returns plain text.
- Whether Red Pen starts directly in pen mode or remembers the last tool.

12. Apple framework references

Implementation references, not product dependencies. Re-check API availability against the deployment target while coding.

Framework / API	Why
Foundation Models	Native language models, structured output, tool calling for task extraction.
SpeechAnalyzer	Modern asynchronous speech analysis/transcription.
Vision RecognizeTextRequest	On-device OCR.
ScreenCaptureKit + sourceRect	Region capture for Screen → Text.
AVCaptureVideoPreviewLayer	Live mirrored camera preview.
NSPanel + nonactivatingPanel	Top-edge auxiliary panel that does not steal the active app.
NSPasteboard.changeCount	Detect general pasteboard content changes.

End state. The first release should feel like six tiny capabilities attached to the top of macOS, not six mini-apps living inside a launcher. The architecture exists to preserve that constraint while allowing the catalog of Dots to grow later.